

CAR — a project report

What Step-Level Verification Certifies, and What It Misses

P. Somnath Reddy

29 September 2026

Contents

1. What the project is	1
2. Why this project, and how the question changed	2
3. The framework	4
4. The papers	5
5. What the project found	9
6. The methodological discipline	18
7. What was refuted, including the project's own claims	19
8. What remains	21
9. An honest assessment	22

What this project is, why it exists, what it read, what it found, and what is left. Compiled 2026-09-29 from the thesis draft, the findings documents, the bibliography and the commit history.

This is a report **about** the project, written for someone who needs to understand and defend it. The thesis itself is docs/thesis/; the lab notebooks are the FINDINGS-*.md files; the ledger of claim against evidence is THESIS.md.

1. What the project is

Title. *What Step-Level Verification Certifies, and What It Misses — a measurement study of selective verification in multi-step LLM reasoning.*

A language model answering a hard question writes a chain of intermediate steps. Any step can be wrong, and a wrong step early on poisons everything after it, because the later steps are generated conditioned on it. If you have a limited budget for checking steps — a calculator, a retriever, a process reward model, another model asked to review the work — where should you spend it?

The obvious design has three parts, and all three have real literature behind them:

1. **A score.** Attach an uncertainty number to each step (token entropy, surprisal, disagreement across resamples).
2. **A calibration.** Turn that score into a threshold using conformal prediction, so you get a distribution-free guarantee instead of a hand-tuned cutoff.
3. **A gate.** Verify the steps above the threshold, subject to a budget, and let the rest through.

This project set out to build exactly that system. It doesn't work. The thesis is an account of *why* it doesn't, and the reasons turn out to be separable, measurable, and more useful than a working system would have been.

The contribution is a characterisation of a design space rather than a method. That's an unusual shape for a thesis, and section 2 explains how it got there.

Scale of the work. 48 commits between 2026-08-31 and 2026-09-28. A 24,620-word draft (front matter, nine chapters, references), 17,008 words of findings documents plus the positioning review and the planning ledger, 28 experiment and utility scripts, 342 tests, 15 figures, 21 bibliography entries. Measurements on 93,129 labelled steps from Math-Shepherd plus a 2,573-step corpus generated for this work.

2. Why this project, and how the question changed

2.1 The original proposal

The project began as a method paper. The proposed contribution was **adaptive conformal calibration under censored feedback**: a gate that verifies only some steps, observes labels only where it verified, and corrects the resulting bias with Bernoulli forced exploration plus inverse-propensity weighting. Each step would be weighted by its downstream influence, so the budget went where an error would do the most damage.

That's a clean idea. It survived about one day.

2.2 The scoop

The second commit in the repository is titled “*Add literature positioning: censored-feedback contribution is scooped.*” A full-PDF literature review (kept as POSITIONING.md) found the core mechanism already published.

Khosravi & Huo, *Conformal Selective Acting* (arXiv:2605.20270, May 2026). Appendix **Theorem E.1** states it precisely: subsample verifier labels with a predictable Bernoulli draw $B_t \sim \text{Bernoulli}(\pi_t)$ where $\pi_t \geq \pi_{\min} > 0$, use the importance-weighted increment $\tilde{X}_t(q) := (B_t/\pi_t) \cdot X_t(q)$, and the supermartingale property survives, every anytime-valid guarantee holds verbatim, and certification delay inflates by at most $1/\pi_{\min}$.

That is forced exploration with $1/\pi$ weighting, under a **stronger** guarantee than this project had hypothesised. This project had guessed at an $O(1/\sqrt{(\varepsilon \cdot T)})$ long-run rate; CSA proves an anytime-pathwise bound. They also characterise the fully-censored case — verify only on accepted rounds and “*the controller stalls at the current threshold but stays valid*” — which is exactly the naive failure mode in this codebase.

The machinery is still implemented and used here (src/car/conformal/adaptive.py). It's cited to CSA, not claimed. Stating that plainly in the introduction is a strength: a reviewer who finds the scoop independently discounts everything else in the document.

2.3 What else died

Over the next week, stress-testing removed the remaining method claims one at a time:

original claim	outcome
Adaptive conformal under censored feedback is novel	scooped (CSA Thm E.1)
Composite token-level uncertainty is the key signal	refuted — measured at AUROC 0.5589
Semantic entropy at intermediate steps is the key signal	refuted — 0.5740; combining buys 0.0002
Influence-weighted allocation beats uniform	refuted — lost on 5 DAG families and 2 real corpora
“Verify early beats verify late”	refuted — worst shape at every scope > 0
StrategyQA as primary benchmark	wrong choice — 72.9% of its graphs are one hop deep
$\alpha = 0.10$ as a working risk target	infeasible — a 32.3% verification entry fee before any method is admissible

By 2026-09-02 the commit log reads “Switch primary benchmark to GSM8K” and then “Commit to the measurement framing.” That’s the pivot.

2.4 What survived, and why it’s defensible

Three things came through the review intact.

Dependent steps within a trajectory. Every conformal gating paper in the comparison treats rounds as independent items: one query, one answer, act or abstain. CSA’s supermartingale lives on a filtration where round t ’s verifier outcome doesn’t change the correctness distribution of round $t+1$. In multi-step reasoning that’s false by construction. Accepting a wrong step at $t=1$ changes the distribution of steps $t=2\dots T$. This isn’t a technicality, it’s the phenomenon the project is about, and it breaks the exchangeability that both CSA and conformal risk control rest on. Nobody has done risk control where the gate’s own accept decision alters the data-generating process downstream. It remains open; this thesis characterises it empirically rather than solving it.

The impossibility floor as an evaluation instrument. Not novel (it’s Kotte’s), but nobody was using it as a reference line for selective verification in reasoning. It costs almost nothing and converts an accuracy-cost plot from “our curve beats theirs” into “here is the achievable region and here is where we sit in it.”

The measurement itself. Which became the thesis.

2.5 The through-line

Everything rests on one distinction, stated in one line:

`global_correct(t) = local_valid(t) AND NOT premise_corrupt(t)`

A step fails two separable ways. **Local invalidity**: the step doesn't follow from its own premises. $47 \times 3 = 131$ is wrong in any context and a calculator catches it every time. **Inherited corruption**: the step follows perfectly from its premises and a premise is false. *Aristotle died in 1850, so he could have used a laptop* is impeccable reasoning to a false conclusion, and no amount of checking the step itself reveals anything.

A verifier reports the first. It's a function of the step plus at most a bounded window of context, with no access to the truth of the premises unless it can independently establish them. Conformal machinery calibrates whatever the verifier reports, so **it calibrates local validity**.

The thesis asks how much of the failure that leaves out. Measured on 93,129 real steps: most of it.

3. The framework

3.1 Why the measurement is possible at all

GSM8K is the benchmark because it uniquely supplies both halves of the decomposition on the same steps:

quantity	how it's observed	instrument
<code>local_valid(t)</code>	does the step's own arithmetic hold?	deterministic evaluation of the <code><<expr=result>></code> annotation
<code>global_correct(t)</code>	does this prefix still reach the right answer?	Math-Shepherd's +/- Monte-Carlo label

Having both signals on the same steps is what turns the local/global gap from a thought experiment into a measurement. The derived quantity the thesis is about:

`inherited_corruption(t) = local_valid(t) AND NOT global_correct(t)`

3.2 Two estimators, and why the difference matters

The headline ratio can be computed two ways:

`C1_all` = $P(\text{inherited}) / P(\text{global error})$ over ALL steps
`C1_checkable` = $P(\text{local_valid} \mid \text{global error, checkable})$

`C1_all` produced the originally-reported 0.6982. It's biased **downward** by the uncheckable rate, because a step with no arithmetic can never enter the numerator but always sits in the denominator. On Math-Shepherd, where 91.4% of steps are checkable, the bias is small and the two agree. On a generator whose steps are 59% prose it collapses to 0.25 and says nothing about reasoning.

C1_checkable is the estimator to quote across generators. This distinction only became visible when a second generator was measured, and getting it wrong would have produced a confident nonsense number.

3.3 Verifier reach

A verifier is characterised by two measurable numbers:

- **scope** — $P(\text{detect} \mid \text{step is globally wrong})$
- **false alarm** — $P(\text{flag} \mid \text{step is globally correct})$

Scope is a property of the verifier *class*, not of the budget.

One modelling decision carries more weight than it looks: in `src/car/verification/scoped.py`, a **missed detection returns SUPPORTED, not INSUFFICIENT**. A verifier that fails to see an error doesn't announce the failure, it says the step looks fine. So a low-scope verifier doesn't merely help less. It feeds the calibrator systematically wrong labels, and the calibrator converges on a threshold that's confident about a risk it can't see.

3.4 The propagation model

Entering corruption and escaping it are asymmetric, and that asymmetry drives every simulated result:

```
CLEAN      -> CORRUPT_1      e · (1 - v_t)          <-- no scope term
CORRUPT_k  -> CLEAN          v_t · scope · decay^(k-1) <-- scope AND decay
```

Entering corruption doesn't depend on the verifier's reach. Escaping it does. A verifier with scope 0 can't reduce corrupted-state occupancy at any budget. $\text{decay} \approx 0.377$ is fitted to Singh & Pawar's measured escape probabilities. The closed form is validated against simulation to within 0.015 over twelve configurations.

Its assumptions are wrong in two known ways, and the *direction* is stated each time. Full repair on detection makes the model optimistic about verification. I.i.d. per-step error is false (later steps are harder), which makes it *conservative* about back-loading — the direction that makes Chapter 6's conclusion safer rather than weaker.

4. The papers

21 entries, verified against the arXiv listing on 2026-09-07. Grouped as the bibliography groups them.

4.1 Conformal prediction and risk control

Vovk, Gammernan & Shafer, *Algorithmic Learning in a Random World* (Springer, 2005). The source of the split-conformal construction. Take scores on a held-out calibration set, compute a finite-sample-corrected quantile $\lceil (n+1)(1-\alpha) \rceil / n$, use it as a threshold. Under exchangeability the acceptance rule has marginal coverage at least $1-\alpha$. The correction is what makes the guarantee hold at finite n rather than asymptotically. *Used as*: the base calibrator in `src/car/conformal/split.py`.

Angelopoulos, Bates, Fisch, Lei & Schuster, *Conformal Risk Control* (ICLR 2024, arXiv:2208.02814). Generalises the conformal guarantee from covering a set to bounding the expectation of any bounded monotone loss. That's the right shape for a verification gate, where

the loss is “a bad step was accepted.” *Used as:* the formal framing for what the gate is supposed to control.

Gibbs & Candès, *Adaptive Conformal Inference Under Distribution Shift* (NeurIPS 2021, arXiv:2106.00170). Relaxes exchangeability by updating the threshold online from observed errors, $q_{t+1} = q_t + \gamma(\alpha - \text{err}_t)$. Buys robustness to shift at the cost of long-run rather than finite-sample validity. *Used as:* the online update this project’s adaptive calibrator is built on.

Barber, Candès, Ramdas & Tibshirani, *Conformal Prediction Beyond Exchangeability* (Annals of Statistics 2023, arXiv:2202.13415). The general treatment. Weighted quantiles give robustness to distribution drift, and a randomization technique admits fitting algorithms that don’t treat data points symmetrically. Provably robust, losing much less coverage under drift while keeping the standard guarantee when data really are exchangeable. *Used as:* the closest available starting point for this thesis’s open problem — and explicitly **not a solution**, because Barber et al. handle *exogenous* drift while the violation here is *endogenous*. The gate’s own decisions move the distribution.

Khosravi & Huo, *Conformal Selective Acting* (arXiv:2605.20270). Controls selective risk with anytime-pathwise validity via a Ville-type e-process per candidate threshold on a Bonferroni grid, giving $R^{\text{act}}_T \leq \alpha + O(\sqrt{(\log(1/\delta))/N_T})$ for all T . Algorithm 1 runs the verifier every round, so the main method has full feedback; the sparse-verifier result is Theorem E.1 in the appendix. Their own empirical table shows validity survives subsampling but utility degrades hard — action rate collapses from 92.8% to 46.7% as π drops to 0.1, and risk falls to 17.8% against a 40% target, badly over-conservative. *Used as:* the paper that scooped this project’s proposed contribution, and the citation for machinery still in the codebase.

Kotte, *When Can Conformal Risk Control Certify LLM Outputs?* (arXiv:2606.29054). **Proposition 3** is the result that should be run before any experiment rather than after: when base risk μ exceeds target α , any distribution-free method must abstain on at least $(\mu - \alpha)/(1 - \alpha)$ of examples. Closed form, checkable in advance, depends on two numbers. Kotte also reports that hard NER/QA/CLS configurations are uncertifiable at $\alpha = 0.10$ and that relaxing to 0.30–0.40 unlocks practical certification. *Used as:* the feasibility instrument throughout — a reference line on every cost plot, and a runtime check in `configs/default.yaml` that refuses to start a run whose budget sits below its own floor. It’s also the direct reason this project’s α moved from 0.10 to 0.30.

4.2 Benchmarks and step-labelled data

Cobbe et al., *Training Verifiers to Solve Math Word Problems* (2021, arXiv:2110.14168). GSM8K: 7,473 train and 1,319 test grade-school word problems with worked solutions. The inline `<<expr=result>>` calculator annotations are what make the local-validity check deterministic — no model, no judge. *Used as:* the primary benchmark.

Geva et al., *Did Aristotle Use a Laptop?* (TACL 2021). StrategyQA: implicit multi-hop reasoning questions shipping **human-annotated decompositions**, which is why it looked like the ideal benchmark for step-level work. *Used as:* the original primary benchmark, and now the subject of Chapter 8’s critique — its decompositions are too shallow to exhibit propagation at all.

Wang et al., *Math-Shepherd* (ACL 2024, arXiv:2312.08935). The dataset the whole thesis measures on, and its construction is what makes the measurement possible. GSM8K and MATH solutions generated by Mistral-7B-SFT, each step labelled + (has the potential to lead to the correct answer) or -. Labels come from Monte-Carlo rollout (“hard estimation”: + if any completion from that prefix reaches the gold answer), not human annotation. Two consequences used throughout. First, the label is **global** correctness — a step inherits - from a corrupted premise even when the step itself is impeccable, which is exactly the quantity this thesis needs and can’t otherwise obtain at scale. Second, the labels are optimistic estimates, so a lucky wrong step can be labelled + and measured global error is a lower bound.

Their own worked example states the phenomenon better than prose can:

Step 2: $13 \times 8 / 13 = <<13*8/13=6>>6$	-	arithmetically WRONG (= 8)
Step 3: $78 - 6 = <<78-6=72>>72$	-	arithmetically CORRECT, inherits
Step 4: $72 - 20 = 52$	-	arithmetically CORRECT, inherits

4.3 Process supervision and step-level scoring

Lightman et al., *Let’s Verify Step by Step* (2023, arXiv:2305.20050). Established that process supervision (scoring each step) beats outcome supervision (scoring the final answer) on mathematical reasoning. *Used as:* the foundation of the PRM line this thesis’s verifiers come from.

Ni et al., *ReProbe* (arXiv:2511.06209). The efficiency result that reshapes the design space. A transformer probe under 10M parameters, reading the **frozen internal states of the generator itself**, outperforms PRMs up to 150× larger and stays competitive with PRMs up to 810× larger. Its labels can come from a larger model or be generated self-supervised, so it needs neither human annotation nor Monte-Carlo consensus filtering.

Two details matter here specifically. The probe’s advantage is largest **out of domain** — on planning and StrategyQA — while strong PRMs reach parity with it *in domain, on MATH and GSM8K*. So on this benchmark a probe should be expected to land near the PRM rather than above it. And Ni et al. report that combining probe and PRM beats either alone, which points at hybrid verifiers rather than replacement. *Used as:* the efficiency bar any “better step score” proposal has to clear, the signal class §7.6 tests directly, and the source of a prediction this thesis made and then missed.

Wen et al., *Embedding Perturbation may Better Reflect Intermediate-Step Uncertainty* (arXiv:2602.02427). Argues that perturbing embeddings reflects step-level uncertainty better than sampling-based agreement does, and that sampling-agreement methods “struggle to pinpoint the intermediate uncertainty.” *Used as:* Chapter 7 measures the sampling-based half and finds it near chance, which is Wen et al.’s conclusion reached from the other direction. Their alternative signal is untouched by this thesis’s negative result, and Chapter 9 names it as future work.

Farquhar, Kossen, Kuhn & Gal, *Detecting Hallucinations Using Semantic Entropy* (Nature 2024). Samples several answers independently, clusters them into meaning-equivalence classes by **bidirectional entailment** under an NLI model, and takes entropy over the cluster distribution. The strongest of the sampling-based signals at whole-answer level. *Used as:* the reference relation §7.2 implements and tests at *step* level, where it does not carry.

4.4 Verification, self-correction and propagation

Huang et al., *LLMs Cannot Self-Correct Reasoning Yet* (ICLR 2024, arXiv:2310.01798). A model asked to check its own reasoning without external feedback doesn't reliably improve and sometimes gets worse. *Used as*: the negative control for Chapter 5, and confirmed there at step level with a measured scope of exactly 0.0000.

Singh & Pawar, *The Hallucination Snowball* (arXiv:2608.14588). Models propagation across agent handoffs as a Markov chain and measures escape probabilities of 24.6%, 48.3% and 89.3% across successive boundaries. *Used as*: the source of this project's decay constant, $\text{decay} \approx 0.377$.

Ro et al., *Sherlock: Reliable and Efficient Agentic Workflow Execution* (arXiv:2511.00330). The closest prior work on the *system* side. Three mechanisms: vulnerability-guided placement (finding error-prone nodes by counterfactual fault injection rather than a structural heuristic like fan-in); cost-optimal verifier selection via a learned per-node selector; and speculative verification, where downstream nodes execute while verification runs in the background. Reports +18.3% accuracy over a non-verifying baseline, 48.7% lower execution time than non-speculative verification, and 26.0% lower verification cost than Monte-Carlo-search placement.

Their motivation is an observation this thesis arrives at independently: “*verifier behavior varies significantly across tasks, and their accuracy-cost relationship is highly non-linear.*” Chapter 2 states the overlap honestly — Sherlock's second mechanism occupies the same ground as Chapter 5, and this thesis doesn't claim that observation as new. What Chapter 5 adds is the *decomposition*: measured on the population deterministic checking provably can't see, reach requires generator-independence **and** task specialisation, and neither alone suffices. That's a distinction a cost-versus-accuracy selector can't make, because both properties are invisible to it.

You et al., *Probabilistic Soundness Guarantees in LLM Reasoning Chains* (arXiv:2507.12948). The closest prior work on the *claim* side, and it tests C1 directly. Their diagnosis is C1 stated independently: “*current LLM-based error detection methods often fail to detect propagated errors because earlier errors can corrupt judgments of downstream reasoning.*” Their remedy is structural — Autoregressive Reasoning Entailment Stability scores each step **solely against previously-verified premises**, inductively, emitting a calibrated soundness score rather than a binary label. Across four benchmarks it reaches 72.1% Macro-F1 (+8.2), and on long synthetic chains it detects propagated errors at **90.3% F1 (+27.6)**.

This is a result the thesis has to accommodate rather than dismiss. It shows the inherited-corruption population isn't *intrinsically* invisible — it's invisible to a verifier reading a step against the generator's **unverified** context, which is what every verifier in Chapter 5 does. So the correct reading of C1 is narrower than “no verifier can see these steps.” Two things stop the numbers being comparable, and the thesis says so: ARES's 90.3% is F1 on their own synthetic ClaimTrees corpus while Chapter 5's 0.9033 is a detection rate on Math-Shepherd's arithmetic-blind subset, and the coincidence is exactly that. And ARES assumes a verified prefix exists, which is precisely what a budgeted gate can't supply.

4.5 Selective labels

Lakkaraju, Kleinberg, Leskovec, Ludwig & Mullainathan, *The Selective Labels Problem* (KDD 2017). Formalises the difficulty when outcomes are observed only for items a policy acted on. A gate that verifies only what it flags observes labels only where it flags, so the label distribution is conditioned on the gate’s own decisions and naive updating doesn’t converge to the right threshold. *Used as:* the reason forced exploration is in the loop at all. Two ordering constraints follow and are enforced in code: the exploration coin must be drawn *before* the gate decision and independently of the score, or the propensity isn’t known; and an INSUFFICIENT verdict must yield **no label**, not a label of “no error.”

4.6 The models

Jiang et al., *Mistral 7B* (arXiv:2310.06825) — the base of Mistral-7B-SFT, the generator behind Math-Shepherd’s solutions, ~45% on GSM8K.

Qwen Team, *Qwen2.5 Technical Report* (arXiv:2412.15115) — Qwen2.5-7B-Instruct is the second generator (Chapter 4, measured at 80.0% GSM8K) and the independent judge (Chapter 5).

Llama Team, *The Llama 3 Herd of Models* (arXiv:2407.21783) — the third generator the thesis names but could not run at draft time, because the model is licence-gated on Kaggle. A run is in progress now.

4.7 A cautionary episode about citations

On 2026-09-07 the project built its bibliography and checked every shorthand name against arXiv. **Four of the seven were wrong.** The project’s own notes had been calling arXiv:2511.06209 “UHeads” when it’s **ReProbe** (Ni et al.); 2511.00330 “Sherlock” in a sense that didn’t match its actual title (*Reliable and Efficient Agentic Workflow Execution*, Ro et al.); 2507.12948 “ARES” when the paper is *Probabilistic Soundness Guarantees in LLM Reasoning Chains* (You et al.); and 2602.02427 “the step-uncertainty paper” when it’s Wen et al. on embedding perturbation.

The *descriptions* in the notes were substantively right — the labels weren’t. Keys are now author-year so a nickname can’t drift from its source again, and `scripts/check_citations.py` asserts that every arXiv ID appearing in the thesis has a bibliography entry. Worth knowing, because “which paper is that?” is a viva question and the honest answer involves this episode.

5. What the project found

5.1 The gap is large, and it widens as models improve (Chapter 4)

Mistral-7B-SFT via Math-Shepherd, 25,971 solutions, 93,129 steps. Every rate is reported per stratum and post-stratified, because Math-Shepherd is a PRM *training* set with a deliberate class mix — its raw +/- balance is a property of their construction, not of the model’s natural error rate.

stratum	solutions	steps	local err	global err	inherited
final answer CORRECT	6,940	21,505	0.0242	0.0000	0.0000
final answer WRONG	19,031	71,624	0.1641	0.7106	0.4961
raw sample (biased mix)	25,971	93,129	0.1305	0.5465	0.3816

$C1_{all} = 0.4961 / 0.7106 = 0.6982$. Under the checkable-conditioned estimator on the same data, **0.7848**. So **78.5% of globally-wrong steps are arithmetically perfect**. Controlling local selective risk at level α bounds nothing about the answer.

Post-stratified to Mistral’s reported 45% GSM8K accuracy: local error 0.1011, global error (μ) **0.3908**.

Corruption is close to absorbing. Of 33,236 steps after the first globally-bad step, 95.9% are still labelled -. Of the 14,573 solutions that *could* have recovered, **0 did** — where “recovered” means every step after the first bad one is good. That zero is the strongest single piece of evidence that the gap can’t be closed by more of the same verification.

The obvious objection, and the answer. Mistral-7B-SFT is a weak 2023 model. Wouldn’t a better generator close the gap by itself? 500 GSM8K test problems were solved by Qwen2.5-7B-Instruct (80.0% measured) and labelled by the same Monte-Carlo procedure, $K = 4$, 8,292 rollouts, 7h40m on two T4s.

	Mistral-7B-SFT	Qwen2.5-7B-Instruct
GSM8K accuracy	~45% (reported)	80.0% (measured)
local error	0.1708	0.0813
global error	0.7106	0.5772
C1 (checkable)	0.7848	0.9040
n globally-wrong checkable steps	46,555	125
95% Wilson CI on C1	[0.781, 0.789]	[0.840, 0.944]

The intervals don’t overlap. **The gap widens.** The mechanism is simple once stated: the stronger model halves its arithmetic slips ($0.171 \rightarrow 0.081$) without halving its inherited corruption, so a larger share of what remains is the kind no calculator can see.

A deterministic verifier becomes *less* useful as the generator improves.

That’s sharper than the thesis originally claimed, and the opposite of what the objection predicted.

What doesn’t transfer. Two quantities turned out to be generator properties stated as task properties, and are now labelled per-generator. Base risk μ falls $0.3908 \rightarrow 0.1221$, which removes the $\alpha = 0.20$ feasibility floor entirely. Corruption persistence falls $95.9\% \rightarrow 66.4\%$, and recoveries go from 0/14,573 to 6/83. Still strongly absorbing, but “near-absorbing” was a Mistral property.

5.2 Reach is semantic, not structural (Chapter 5)

If a verifier misses inherited corruption because it sees too little context, widening the lookback should help. On 35,535 real steps:

lookback k	scope	cost per call
0 (step-local)	0.0000	1.00
1	0.1197	1.85
3	0.1880	2.73
∞ (unbounded)	0.1999	3.01

Step-local arithmetic detects nothing, by construction. Widening the window has sharply diminishing returns and triples the cost. **The ceiling is structural:** 28,434 of the 35,535 steps — 80.0% — have *no upstream arithmetic error at all*. The mistake is in the setup, not the calculation.

That defines the real question: on the steps arithmetic provably can't see, what scope does a *semantic* verifier achieve? Four classes, measured on Kaggle:

verifier	independent?	task-trained?	scope	false alarm
arithmetic, unbounded look-back	yes	—	0.1999	—
same-model critic (the generator)	no	no	0.0000	0.0000
independent judge (Qwen2.5-7B)	yes	no	0.2283	0.0200
task PRM (Math-Shepherd-7B)	yes	yes	0.9033	0.0987

The same-model critic detects **zero** errors: it's the model that wrote these solutions, asked whether they're sound, and it approves everything. That's Huang et al. measured directly at step level.

Reach isn't how far back you look. It requires a verifier that is independent of the generator **and** specialised for the task. Neither alone suffices.

The original specification listed calculator, retrieval and sandbox as interchangeable reliability mechanisms. They span 0.00 to 0.90 scope.

The number that turns out to matter most isn't the 0.9033. It's the 0.0987 next to it — and Chapter 7 is where that becomes clear.

5.3 “Verify early” is false (Chapter 6)

The original hypothesis H3: verify early, because an error caught at step 1 costs one call while the same error at step 5 has already contaminated four downstream conclusions. The intuition about *cost* is right. The inference about *allocation* is wrong.

Front-loading is the worst allocation shape at every scope above zero, tested on linear chains, five synthetic DAG families, all 2,272 StrategyQA graphs, 6,974 derived GSM8K graphs, and the corrected GSM8K graphs. Broken out by depth on GSM8K:

depth	graphs	uniform	front	back	influence	depth	spread
2	2,988	0.2049	0.2049	0.2042	0.2033	0.2048	0.0033
4	1,010	0.2992	0.3269	0.2577	0.3241	0.2455	0.0814
6	52	0.3578	0.4059	0.2872	0.4013	0.2779	0.1279

The spread grows monotonically with depth, and front and influence diverge from the field in exactly the regime the thesis is about. At depth 6, front-loading costs 12.8 points of final accuracy against ancestor-count allocation.

The last defence closed by measurement. Early steps might be intrinsically harder, which would restore the case for front-loading. They aren’t. Local error rises from 0.1095 at step 1 to 0.2182 at step 8, $\text{corr}(\text{position}, \text{local error}) = +0.950$. Later steps are *harder*, which favours back-loading further than the model already did.

Influence weighting is dead, and the first test couldn’t have shown it. The first test ran on linear chains, where the descendant count of step t is exactly $T - t$ — so the influence schedule is a monotone decreasing function of position, correlating -1.000 with the front schedule. The test was rejecting front-loading and reporting it as a rejection of influence weighting. Caught by a direct challenge to test tree topologies before dropping the claim, and it’s the single most useful methodological correction in the project. Re-run on branching structures, influence weighting is distinguishable — and still loses to plain uniform everywhere. The replacement finding: the useful structural signal is **ancestor count**, not descendant count.

The dependency-graph audit found a bug, and not the one it was looking for. 50 graphs were adjudicated, stratified by whether they contain an ambiguous link. The audit was queued to quantify a known 9.6% ambiguity caveat. It instead exposed a systematic *recall* failure: the operand regex was `-?\d+(?:\.\d+)?`, and the optional sign **absorbed the subtraction operator**, so “110-80” produced [110, -80], -80 matched no earlier result, and the dependency vanished. **Every subtraction in the corpus silently lost its edge.**

metric	before	after
mean longest path	2.54	2.79
steps with a non-terminal descendant	26.6%	29.9%
shape classified other	22.9%	5.9%

The collapse of the other category is the strongest evidence the fix is right: those graphs were *fragmented* into unclassifiable pieces by the missing edges. It also overturned a published conclusion — an earlier version reported that the best structural signal is benchmark-dependent, which was an artifact. With corrected graphs **depth wins on both**. The measured edge error rate is **5.7%** (16.5% in ambiguous graphs, 4.2% elsewhere).

The caveat you set out to quantify is rarely the one that matters. Auditing a derivation is worth more than characterising its known limitation.

5.4 The assembled gate fails, at three points (Chapter 7)

The core chapter, and the longest. 500 Qwen solutions, 2,573 steps, split 175 dev / 143 calibration / 182 test by hash of example id.

Point one: the signal doesn't rank the risk.

AUROC for detecting a globally-wrong step:

```
0.5589  token-level only
0.5740  semantic divergence only
0.5742  both
```

Combining them adds **0.0002**. The two correlate at $r = +0.44$, so they're largely the same information. This isn't a harness failure: the same pipeline on synthetic features with one-standard-deviation separation gives AUROC **0.8668**, and on pure noise **0.4828**.

The equivalence relation was load-bearing, and nearly produced a wrong paper. Clustering resampled continuations requires deciding when two steps mean the same thing. The library default was string equality — and Qwen writes one computation three ways ($48/2 = <<48/2=24>>24$ clips, $\backslash(48 / 2 = 24 \backslash)$ clips, That gives $48 / 2 = 24$ clips.). Under string equality those are three meanings. Exact match inflates divergence by half and drives AUROC *below chance* (0.4904). **Had the default been used, this chapter would have reported that semantic divergence is anti-predictive** — a clean sub-chance AUROC reads as “this signal is actively misleading” rather than “your clustering is broken.”

The obvious objection is then that the cheap relation *is* the result. So the reference relation was implemented: bidirectional entailment under microsoft/deberta-large-mnli, on the same committed samples.

relation	mean divergence	AUROC test
numeric equivalence	0.4239	0.5740
exact string match	0.6468	0.5287
bidirectional entailment	0.1205	0.5625

Entailment scores **0.5625**, CI [0.512, 0.614]. The 0.0114 gap to numeric equivalence is *not* a ranking — a solution-clustered bootstrap puts it at $[-0.087, +0.059]$. The careful claim is the other

one: the three relations **nest** on one permissiveness axis, calling 0.1%, 31.9% and 78.0% of pairs equal, and across that entire range the measurement doesn't move.

A finding inside the finding: **entailment turns out to be the most permissive relation, not the strictest**. 86.6% of 27,936 directed pairs are judged entailing. On pairs where numeric equivalence *disagrees*, the NLI model still calls 52.0% mutually entailing — it checks whether two sentences are about the same thing, not whether they compute the same number.

Point two: the calibration certifies the wrong quantity. Base risk on test steps is 0.1578, so only $\alpha < 0.1578$ tests anything at all.

α	binds?	split conformal risk	verify %
0.05	yes	0.1491	4.6%
0.10	yes	0.1494	9.2%
0.15	yes	0.1516	14.1%
0.30	no	0.1538	21.8%

At $\alpha = 0.05$ measured selective risk is **three times the target**. Risk barely moves across the whole sweep (0.1491 $\rightarrow$ 0.1587) while verification climbs from 4.6% to 26.9%. **The gate spends budget and buys nothing.**

The reason is that split conformal fits the threshold so the acceptance region *covers* $1 - \alpha$ of correct steps. That's coverage. When the score is uninformative the accepted region contains wrong steps at approximately the base rate wherever the threshold sits — the guarantee holds perfectly and the quantity of interest is untouched, and **nothing in the procedure reports that**.

Point three: the best verifier available is net-negative. Projected final-answer accuracy against a no-gate baseline of **0.8022**:

verifier	scope	FA	always verify	split conformal
independent judge	0.2283	0.0200	0.8022	0.8022
task PRM	0.9033	0.0987	0.7637	0.7912
task PRM, FA = 0 ablation	0.9033	0.0000	0.9231	0.8681

The highest-scope verifier loses 4 points, and more verification makes it worse. Zero out its false alarms and the same verifier gains 12.

The oracle settles which component is at fault, and it isn't the verifier. Running the identical calibrator, budget and verifier behind a score that reads the label:

condition	verify %	calls/q	selective risk	PROJ accuracy
no gate	0.0%	0.00	0.1554	0.8022
split conformal, $\alpha = 0.30$	21.8%	1.09	0.1538	0.7912
probe score	19.1%	0.96	0.1394	0.7802
oracle	7.3%	0.37	0.0885	0.9780

A perfect score cuts selective risk by 43% *and* spends a third of the verification. The same verifier at the same 9.87% false-alarm rate moves from 0.7637 to 0.9780, seventeen points above baseline.

The figure of merit isn't $\text{scope} - \text{FA}$. It's conditioned on what the gate selects: $\text{scope} \times P(\text{wrong} \mid \text{verified})$ against $\text{FA} \times P(\text{correct} \mid \text{verified})$. The score sets that conditioning. The PRM isn't a bad verifier being oversold, it's a good verifier being aimed badly.

That's a correction to how Chapter 5's headline should be stated, and the thesis records it as one.

But the oracle still misses $\alpha = 0.05$ by 1.8 $\times$. At two calls per question over a mean of 5.15 steps, most wrong steps go unverified however perfectly they're ranked. So there are **two** bottlenecks, and they're separable: the score is worth $0.154 \rightarrow 0.089$, and the budget is what stands between 0.089 and 0.05. The third failure point isn't independent — it's downstream of the first.

5.5 A better signal exists, and it isn't enough (§7.6)

ReProbe makes the cheapest available test: does a probe on the generator's own frozen hidden states rank what the measured signals can't? One teacher-forced pass, hidden states at each step's final token across all 29 tensors, a logistic probe per layer, same hash splits.

AUROC 0.6968 on test, against 0.5742 for the best measured signal. So the §7.2 result is about *those signals*, not about step-level uncertainty in general.

Two things stop it being over-read, and both are instructive.

The **winner's curse is large and visible**: the selection-split AUROC is 0.8748 against 0.6968 on test, a **0.18 gap**, from choosing the best of 29 layers $\times$ 5 regularisation strengths on 287 selection steps. Had the layer been chosen on test, this section would report ~ 0.87 and claim the probe beats the Chapter 5 PRM. `select_and_fit` doesn't take test indices as a parameter and a test asserts its signature can't grow one.

And **an earlier draft called 0.6968 a floor on evidence that couldn't support it**. The claim rested on a learning curve climbing $0.7374 \rightarrow 0.8748$ — evaluated on the *selection* split, the same data the layer and regularisation were chosen on. Measured properly, at fixed layer and C, doubling training data from 670 to 1,361 steps is worth **+0.0105**, CI **[-0.035, +0.057]**. Positive, and nowhere near what the leaked curve implied. The claim is recorded as **unsupported** rather than refuted: its evidence was invalid, which isn't the same as being false.

AUROC was also the wrong thing to train on. §7.4 showed the projection only pays for the **first** globally-wrong step, and this probe correlates *positively* with step position — it spends its ranking power on late steps no repair can rescue. Retraining on the first-bad label:

trained on	AUROC	first-bad recall	score-position corr
global label, dev only (published)	0.6968	0.3000	+0.233
the first-bad label	0.5735	0.4500	−0.472

Training on the right target raises the quantity that pays and **removes the AUROC advantage entirely** — 0.5735 sits level with the 0.5742 baseline whose failure is the chapter’s central result. Two objectives that trade against each other, and only one is what the system is paid on. The gain is significant against one reasonable comparator (+0.2216, CI [+0.051, +0.390]) and not the other (+0.1412, CI [−0.065, +0.333]), and the thesis reports both rather than the flattering one.

The instrument lever is null. Held at matched layer, a non-linear head is worth +0.0051, CI [−0.031, +0.042]. Across four matched-layer comparisons the *sign flips*, and the only interval excluding zero favours the **linear** probe.

And the reason is the most useful finding in the section. Layer choice isn’t resolvable at this sample size: 287 selection steps separate the top five candidate layers by at most 0.017 while their held-out AUROCs differ by up to **0.075**, a 4.3× ratio. The pooled arm shows the cost directly — it chose layer 28 (selection 0.8667, test 0.6896) over layer 19 (selection 0.8655, test **0.7645**). **A 0.0012 margin on selection bought a 0.0749 loss on test.**

The probe series has reached the resolution limit of this corpus. The binding constraint isn’t which probe, how much training data, or what target — it’s that 287 selection steps and 925 test steps can’t separate effects of this size.

5.6 How good does the score need to be? (§7.4)

Score quality can be made a dial. Under the binormal model a wrong step draws from $N(d, 1)$ and a correct step from $N(0, 1)$, so $AUROC = \Phi(d/\sqrt{2})$ and the separation inverts in closed form as $d = \sqrt{2} \cdot \Phi^{-1}(AUROC)$. Everything else held fixed — same corpus, splits, calibrator, budget, verifier — so only the ranking moves. 13 grid points × 64 seeds.

AUROC	sel. risk	1st-bad recall	PROJ acc	
0.550	0.1536	0.2853	0.7782	net −
0.625	0.1463	0.3786	0.7940	net −
0.650	0.1441	0.4111	0.8010	~same
0.700	0.1405	0.4635	0.8119	net +
0.990	0.1258	0.7736	0.8712	net +

The crossing is at AUROC ≈ 0.65 — below the probe that already exists. An earlier draft guessed “well above 0.70,” which was wrong in the direction that matters.

The threshold is made entirely of false alarms. Rerun with the verifier’s false-alarm rate switched off and there’s no crossing to find: at FA = 0 the PRM is worth having at *every* score quality tested, down to 0.55. Halving a verifier’s false-alarm rate lowers the score quality you need more than raising its detection rate does.

And no score quality holds a binding α . The best any row manages at $\alpha = 0.05$ is 0.0956, at AUROC 0.99 — still $1.9\times$ the target. Score quality isn’t the binding constraint on risk control. The budget is.

The sweep also refutes the metric it’s built on. Pinning verification at 19.1% so only ranking differs:

	AUROC	1st-bad recall	PROJ acc
synthetic score	0.6974	0.3413	0.8206
probe, layer 25	0.6968	0.3077	0.7802

The probe sits at the **0th percentile of 64 synthetic draws** at the same AUROC. It catches slightly *more* wrong steps and rescues *fewer* answers, because $\text{corr}(\text{position}, \text{probe}) = +0.1813$ against the composite’s -0.2766 . The probe flags late steps; only the first bad step can be repaired.

Two scores with identical AUROC are worth different amounts. Rank by AUROC if you must compare to the literature; select on first-bad-step recall.

5.7 Feasibility and the benchmark critique (Chapter 8)

The risk target is constrained before any method is chosen. With $\mu = 0.3908$, Kotte’s floor at $\alpha = 0.10$ charges **32.3%** of the entire budget as an entry fee. The specification’s $\alpha = 0.10$ wasn’t a modest target; against global risk it’s close to infeasible. Config moved to $\alpha = 0.30$, and `configs/default.yaml` now enforces the floor at setup.

But the floor is a property of the **generator**: on Qwen ($\mu = 0.1221$), $\alpha = 0.20$ carries no floor at all. Any statement about attainable α must name its model. And there’s a sting — when μ falls below α the target becomes *vacuous* rather than easy. At $\alpha = 0.30$ against $\mu = 0.1578$, verifying nothing satisfies the guarantee. A conformal result satisfied by the empty policy certifies nothing.

StrategyQA can’t exercise the phenomenon it’s used for. All 2,272 annotated decompositions:

property	StrategyQA	GSM8K (corrected)
mean graph depth	2.30	2.79
graphs exactly one hop deep	72.9%	—
steps with a non-terminal descendant	11.2%	29.9%
max depth	5	8

A step can only corrupt downstream reasoning if downstream reasoning exists. The phenomenon this thesis is about is nearly absent from the benchmark the specification chose to study it on. The consequence is visible in the allocation results: on StrategyQA the best-to-worst policy spread is 0.0126, which is measuring noise.

Run the feasibility test and the benchmark-capacity test as experiment zero. They’re the cheapest experiments in the project and they constrain everything that follows.

6. The methodological discipline

This is arguably the project’s most defensible feature, and it’s worth being able to speak to.

6.1 Errors caught by recomputation

Roughly **75 numerical and analytical errors** were found and fixed across several adversarial verification passes. Several were the same class of mistake:

Three tokenizer faults in Chapter 5 each produced a confident wrong answer. The first completed run reported the *opposite* conclusion — negative net scope — because the PRM flagged 94.9% of the control group, which are steps carrying its own training labels. Cause: `tok.encode` yielded `_+=648` locally but `+=28806` on Kaggle, the same characters without the SentencePiece word-boundary marker, so the PRM was read at vocabulary indices it had never been trained to emit at. A second fault hardcoded an *input* id that Kaggle encoded differently, so the position mask matched nothing and every score came back NaN. A third was right padding in batched decoder-only generation, which moved the Qwen judge from an untrustworthy 0.1933 to 0.2283.

The fix is a **validation gate**: score 400 steps with known labels *before* the real measurement and abort below 0.15 separation. It caught three successive broken configurations before the fourth passed at 0.5788.

On a borrowed model, the harness must prove it can reproduce that model’s known behaviour before any novel number from it is believed.

Denominator errors, found four separate times. The entailment result was initially conditioned on the NLI model’s own verdict. Probe round two compared different layers and attributed the difference to data. The non-linear probe repeated it. And §4.3 once copied a denominator from the wrong table.

The sampling trap. The Math-Shepherd file is sorted into contiguous blocks by label, so **any prefix read is close to single-class** — the first 80 MB is 100% wrong-answer GSM8K. It cost two debugging rounds to find and came back a third time when a `limit=` parameter reintroduced it. `download_data.py` now fetches 48 strided range requests and asserts the class balance, and a test asserts a prefix read is more skewed than a strided one.

6.2 Guards that stop numbers drifting

guard	what it prevents
<code>check_citations.py</code>	a cited arXiv id with no bibliography entry
<code>check_prose.py --baseline/--compare</code>	an editing pass silently moving a number or flipping a hedge
figures 7.1–7.3 computed at render time	a plot disagreeing with the table above it
regression tests on refuted claims	a refutation quietly stopping reproducing
annotation-rate and solve-rate gates	a rate measured on a biased subset of steps
feasibility check at config time	a run whose budget sits below its own Kotte floor
<code>select_and_fit</code> can't take test indices	the winner's curse becoming the headline
hash-based splits	an example migrating between splits as data grows

342 tests, no GPU and no network required.

6.3 Measured versus modelled

The thesis reports two kinds of number and never blends them. **Measured** quantities come from real model output with real labels. **Modelled** quantities come from the propagation model, whose closed form is validated against simulation to within 0.015 over twelve configurations and whose assumptions are known to be wrong. Every modelled number is labelled PROJECTED, and where the assumption affects a conclusion the direction of the bias is stated.

This matters most in Chapter 7: final-answer accuracy under gating **cannot** be measured on a fixed corpus, because the gate can't change what the model wrote. A regression test pins that with scope 0 and no false alarms the projection reduces *exactly* to observed accuracy. A projection that moves when nothing was repaired is fabricating.

7. What was refuted, including the project's own claims

The thesis refuted more claims than it established, and most of the casualties were its own. Every row that pins a measurement carries a regression test.

claim	source	outcome
Adaptive conformal under censored feedback is novel †	this project’s spec	scooped
Composite token-level uncertainty is the key signal	spec	refuted — AUROC 0.5589
Semantic entropy at step level is the key signal	spec	refuted — 0.5740
Influence weighting beats uniform	spec	refuted — loses on 5 DAG families, 2 corpora
Verify early beats verify late	spec	refuted — worst at every scope > 0
$\alpha = 0.10$ is workable	spec	close to infeasible on Mistral
StrategyQA is a suitable primary benchmark	spec	wrong choice
The best structural signal is benchmark-dependent	this thesis, earlier draft	overturned — extraction-bug artifact
Corruption is near-absorbing net = scope – FA is the figure of merit †	this thesis, ch. 4 this thesis, ch. 5	generator-specific corrected — base-rate error
A calibrated gate controls risk	the whole premise	refuted — misses α by 3×
The three failures are independent †	ch. 7 first draft	partly wrong — verifier is downstream of score
Internal states carry no usable step signal	implied by ch. 7 first draft	refuted — probe reaches 0.6968
A probe here will land near 0.9033	ch. 9 prediction	missed — 0.6968
0.6968 is a floor pending more data	ch. 7.6	unsupported — curve was on the selection split
A non-linear probe is the right instrument	ch. 7.6, ch. 9	null — +0.0051, sign flips by layer
The verifier needs a score “well above 0.70”	ch. 7 earlier draft	wrong, and low — crossing is ≈ 0.65
AUROC is the figure of merit for a step score	implicit throughout	refuted — first-bad recall is
The cheap relation was hiding the signal	the obvious objection to §7.2	refuted — entailment 0.5625

† Three rows have no regression test, because what they refute isn’t a number that could move: one is a literature fact, two are corrections to how a quantity should be *interpreted*.

One distinction worth being able to explain in a viva. C9c was downgraded from “refuted” to “**unsupported**.” Its supporting curve was scored on the selection split, so the evidence was invalid. A clean test gives +0.0105 with CI [−0.035, +0.057] — an interval that neither establishes the claim nor rules it out. Invalid evidence isn’t the same as a false claim, and the thesis marks the difference.

8. What remains

8.1 Known gaps, in the order an examiner would care

1. **No related-work chapter separate from background.** For a paper submission these would split. Cheap to fix.
2. **More first-bad-step labels.** §7.6 trains a score against the right target and it works, but on 49 training positives and 40 test ones. Only 108 such steps exist in the whole corpus, and that’s the binding constraint on the section’s only constructive result.
3. **More evaluation data generally.** All three probe levers sit inside their own intervals. The selection split separates candidate layers by 0.017 while their test AUROCs differ by 0.075, so an arbitrary layer choice swamps every effect the section can measure. The open question isn’t which probe, it’s how much evaluation data.
4. **A third generator.** Would settle which per-generator quantities (μ , absorption persistence, the position gradient) are monotone in model strength and which are idiosyncratic. **This is currently running** — see 8.3.
5. **ARES-style conditioning under a budget.** The clearest scientific gap. You et al. detect propagated errors at 90.3% F1 by scoring each step against *previously-verified* premises, which is the structural change C1 implies is necessary. It assumes a verified prefix, which is exactly what a budgeted gate can’t supply. What happens when only a fraction of the prefix is verified — and how the gate should choose *which* fraction — is the most promising question this thesis can hand on. It also reframes Chapter 6: allocation would stop being about catching errors and become about building a trustworthy prefix, for which ancestor count is a far more natural signal than it is for the objective actually tested.
6. **Risk control under gate-induced dependence.** The theoretical problem is still open. Barber et al. relax exchangeability for *exogenous* drift; the violation here is endogenous. Their weighted-quantile machinery is the closest starting point, not a solution. This thesis characterises the phenomenon and doesn’t solve it.
7. **Cross-domain replication.** Everything measured here is grade-school arithmetic. Whether the gap has the same size on code, multi-hop retrieval or agentic tool use is untested. The mechanism has no obvious arithmetic dependence, but that’s an argument, not a measurement.

8.2 Limitations that are inherent rather than unfinished

- **A “step” isn’t a model-invariant unit.** Math-Shepherd’s step is one calculator operation; Qwen writes 5.15 steps per solution of which 59% are narration asserting no arithmetic. Per-step rates across generators are rates over different units. No amount of better extraction fixes this.
- **Label semantics.** Math-Shepherd’s +/- are Monte-Carlo estimates of “leads to a correct answer,” not proofs, so measured global error is a lower bound.
- **Derived graphs.** 5.7% edge error is itself a *lower* bound — dependencies routed through unannotated solution lines are invisible to any operand-matching scheme. Adjudication was by LLM, not blind human annotation.
- **Sample size in the end-to-end run.** 182 test questions, 925 test steps. The headline gap is far outside sampling noise; between-condition differences aren’t.

8.3 In flight right now

A **Llama 3.1 8B Instruct** generator-transfer run is executing on Kaggle (kernel somnath26/car-llama-transfer, T4 × 2). This is gap #4, and it was blocked at draft time because the model is licence-gated — the Kaggle API returned “*User has not consented to terms of use*” and silently didn’t mount the model, ending two sessions at setup.

Status as of this report: generation is finished at a **68.4% solve rate** (inside the 0.55–0.95 band, so the prompt is being followed), and 5,752 global rollouts are in progress.

One gate failed, and it’s informative rather than fatal. **Annotation rate 0.3101 against a 0.60 floor** — Llama writes `<<expr=result>>` markers on only about a third of its steps. That’s the same gate that caught Qwen writing LaTeX. It affects only the arithmetic-checking half of Chapter 4, because μ and persistence come from the rollouts, which never parse step text. So the quantities the run was launched for will land cleanly. C1 for Llama will need the notation=“any” path, and how much that recovers is a measurement still to be made.

8.4 Submission-level gaps

The front matter carries placeholders for university, supervisor and degree. The LaTeX build exists (`scripts/build_latex.py`, validated statically by `check_latex.py` since no TeX toolchain is installed on this machine) and produces a compilable project, but it hasn’t been compiled end to end.

9. An honest assessment

What’s genuinely strong. The measurement apparatus, and the discipline around it. Hash-based splits that can’t leak, a null-hypothesis instrument that proves the pipeline detects signal when signal exists, validation gates on borrowed models, regression tests that pin refuted claims, a projection that provably reduces to observed accuracy when nothing is repaired, and a documented habit of recomputing rather than trusting. Roughly 75 errors were caught this way, several of which would otherwise have produced confident, publishable, wrong results. The three tokenizer faults in Chapter 5 are the clearest case: without the validation gate the project would have published a negative result produced entirely by a tokenizer mismatch.

What’s genuinely interesting. Three things that transfer beyond this project. That a deterministic verifier gets *less* useful as generators improve, which is counterintuitive and measured with disjoint intervals. That AUROC is the wrong figure of merit for a step-level score under propagation, demonstrated by two scores of identical AUROC differing by 0.04 projected accuracy. And that a verifier’s *false-alarm* rate, not its detection rate, sets how good your score has to be — at $FA = 0$ the crossing disappears entirely.

What’s weak, and should be said first rather than defended. The end-to-end corpus is small: 182 test questions, 925 test steps, 108 first-bad steps. §7.6’s entire constructive half sits inside its own confidence intervals, and the thesis says so rather than ranking effects it can’t separate. The second generator contributes only 125 globally-wrong checkable steps. Everything is grade-school arithmetic. And the headline C1 number moved from 69.8% to 78.5% during the project, not because a measurement changed but because the first estimator turned out not to travel across generators.

The shape of the contribution. This is a negative result with a mechanism, not a null. That distinction is what makes it defensible: the gate doesn’t merely fail to work, it fails at locatable points, two of which are genuinely independent (the signal and the budget) and one of which turned out to be downstream of the first. Each leg has a number and a regression test, and the whole thing is falsifiable in the only way that matters — a signal that ranks step error would break it.

The project is more useful for having failed. A working gate on GSM8K would have been one more entry in a crowded area. A measurement of *why* the obvious design can’t work, with the failure points separated and quantified, is a result that transfers.